EXP-Bench: Can AI Conduct AI Research Experiments?

Taro Yamada

Abstract

Most benchmarks assess isolated skills like code generation or analysis, but not the end-to-end rigor of scientific experimentation required for AI research. We introduce EXP-Bench, the first benchmark to evaluate AI agents on complete, executable AI research experiments. EXP-Bench comprises 461 tasks from 51 peer-reviewed NeurIPS/ICLR 2024 papers, spanning computer vision, natural language processing, reinforcement learning, and applications. Each task provides a research question, high-level method, and masked code repository; ground truth includes design specifications, implementation diffs, and validated conclusions. Our evaluation uses conjunctive multi-metric scoring across Design (D), Implementation (I), Execution (E), and Conclusion (C) phases, with execution validation in Docker containers. Across seven agent-model pairs, we find that while agents achieve partial correctness on individual components, they struggle with end-to-end completion: the best performer (OpenHands + o3-mini) achieves only 18.4% design accuracy, 20.3% implementation accuracy, and 0.5% complete end-to-end success. Our analysis reveals systematic failure modes including missing essential implementation components (39.7%), environment configuration errors (29.4%), and incorrect conclusions (19.7%). EXP-Bench provides a rigorous testbed for advancing AI research automation capabilities.

1 Introduction

The automation of scientific research represents a frontier challenge for artificial intelligence, requiring agents to design experiments, implement methodologies, execute code, and interpret results [1, 2]. While substantial progress has been made in evaluating AI systems on isolated tasks—code generation [5], mathematical reasoning, and text analysis—existing benchmarks fail to capture the end-to-end rigor and complexity of conducting complete scientific experiments.

Current AI evaluation frameworks assess discrete capabilities rather than integrated workflows. Code generation benchmarks like HumanEval focus on implementing individual functions, while agent benchmarks like AgentBench [4] evaluate task completion in simplified environments. However, real AI research demands the orchestration of multiple complex steps: formulating experimental designs, implementing sophisticated algorithms, managing computational environments, executing experiments, and drawing valid scientific conclusions from results.

This gap between current evaluation and real research needs motivates our work. We introduce EXP-Bench, the first benchmark designed to evaluate AI agents on complete, executable AI research experiments. Our key insight is to anchor evaluation tasks in peer-reviewed publications and their open-source implementations, providing realistic research questions paired with execution-validated ground truths.

EXP-Bench comprises 461 research tasks semi-automatically extracted from 51 influential NeurIPS and ICLR 2024 papers, spanning computer vision, natural language processing, reinforcement learning, and applications. Each task provides: (1) a research question derived from published work,

Preprint. Under review.

(2) a high-level methodological approach, (3) a masked code repository with answer-revealing files removed, and (4) ground truth encompassing design specifications, implementation details, and validated experimental conclusions.

Our evaluation protocol employs conjunctive multi-metric scoring across four phases: Design (D), Implementation (I), Execution (E), and Conclusion (C). We validate implementations by executing agent-generated code in isolated Docker containers and employ an LLM judge to assess design and conclusion quality. This rigorous approach reveals the gap between component-wise and end-to-end performance.

We evaluate seven agent-model combinations, including OpenHands [6] with various language models (o3-mini, Claude-3.7 Sonnet, Amazon Nova Pro, etc.) and IterativeAgent with multiple models. Our results demonstrate that while agents achieve moderate success on individual components—with the best performer reaching 35.0% implementation accuracy—end-to-end success remains elusive, with complete task success rates below 1% across all configurations.

Our contributions include: (1) EXP-Bench, the first benchmark for evaluating complete AI research experiments; (2) a scalable semi-automated pipeline for extracting executable research tasks from publications; (3) a comprehensive evaluation of current AI agents revealing systematic limitations; and (4) a detailed failure taxonomy identifying specific bottlenecks in research automation.

2 Related Work

AI Agent Evaluation. Current benchmarks for AI agents focus on discrete tasks rather than integrated workflows. AgentBench [4] evaluates agents across diverse environments but emphasizes task completion rather than scientific rigor. Similarly, other agent benchmarks assess capabilities like tool use, planning, and interaction but do not capture the complexities of conducting complete research experiments. Our work extends this evaluation paradigm to the domain of scientific research automation.

Code Generation and Programming Benchmarks. Substantial effort has been devoted to evaluating code generation capabilities through benchmarks like HumanEval and MBPP, with recent advances in models like CodeLlama [5] and others. Repository-level evaluation has been explored in RepoBench [3], which assesses code completion in realistic development contexts. However, these benchmarks focus on programming competency rather than the scientific experimental process, missing crucial aspects like experimental design, result interpretation, and scientific validity.

Research Automation and Scientific AI. The automation of scientific discovery has garnered increasing attention, with efforts like SciAgents [2] exploring multi-agent systems for scientific reasoning and ResearchAgent [1] investigating automated research processes. However, most prior work focuses on specific aspects of research (e.g., hypothesis generation, literature review) rather than complete experimental workflows. Additionally, these systems often lack rigorous evaluation protocols that validate both the correctness of generated implementations and the validity of scientific conclusions.

Benchmarks for Complex Reasoning. Several benchmarks evaluate multi-step reasoning and complex problem-solving, but they typically focus on mathematical or logical reasoning rather than empirical scientific methodology. While these benchmarks assess important cognitive capabilities, they do not capture the unique challenges of scientific experimentation: managing computational environments, implementing algorithms from papers, executing experiments, and interpreting results within the context of existing research.

EXP-Bench addresses these gaps by providing the first benchmark that evaluates AI agents on complete, executable research experiments anchored in peer-reviewed scientific literature. Our approach uniquely combines methodological rigor, execution validation, and comprehensive evaluation across all phases of the experimental process.

3 The EXP-Bench Benchmark and Dataset

EXP-Bench evaluates AI agents on their ability to conduct complete AI research experiments. Figure 1 illustrates our approach: given a research question and high-level methodology from a peer-reviewed

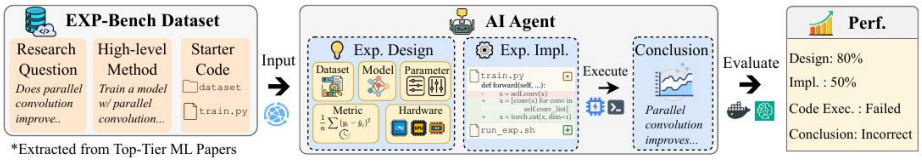


Figure 1: EXP-Bench evaluates AI agents on research experiment tasks extracted semi-autonomously from peer-reviewed AI papers. Agents receive masked repositories and must complete the full experimental workflow.

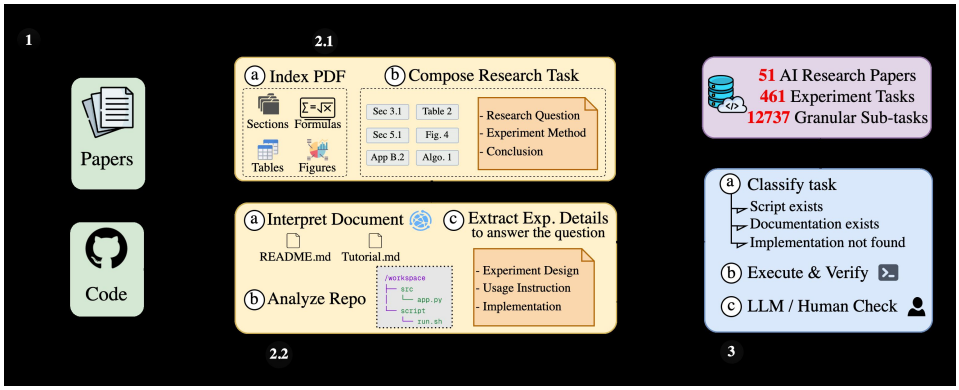


Figure 2: EXP-Bench semi-automated dataset construction pipeline extracts research tasks from peer-reviewed papers and validates their executability.

paper, agents must design experiments, implement algorithms, execute code, and draw scientific conclusions. This section details our benchmark design, data construction pipeline, and evaluation protocol.

3.1 Benchmark Design

Task Structure. Each EXP-Bench task comprises four components: (1) *Research Question*: A specific experimental objective derived from published work; (2) *High-level Method*: A methodological approach without implementation details; (3) *Masked Repository*: A code repository with answer-revealing files removed; and (4) *Ground Truth*: Validated design specifications, implementation diffs, and experimental conclusions.

Evaluation Phases. We assess agent performance across four phases corresponding to the complete research workflow:

- **Design (D):** Experimental design specification including variables, metrics, and methodology
- **Implementation (I):** Algorithm implementation and code generation
- **Execution (E):** Successful code execution producing results
- **Conclusion (C):** Scientific interpretation of experimental outcomes

Conjunctive Evaluation. Unlike traditional benchmarks that assess isolated capabilities, EXP-Bench employs conjunctive scoring that requires success across multiple phases. We report both individual phase accuracy (D, I, E, C) and conjunctive metrics (I·E for implementation *and* execution, All✓ for design *and* implementation *and* conclusion, All·E✓ for complete success including execution).

3.2 Data Construction Pipeline

Our semi-automated pipeline extracts executable research tasks from peer-reviewed publications through three stages, illustrated in Figure 2.

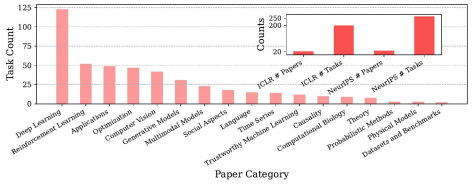


Figure 3: EXP-Bench's dataset comprises tasks from a diverse set of ML research categories.

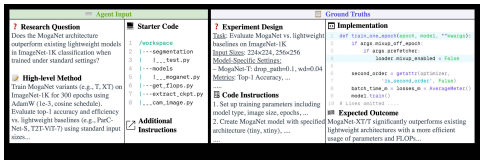


Figure 4: One AI research task example from ICLR 2024 MogaNet research demonstrating the complete experimental workflow.

Stage 1: Source Selection. We select influential NeurIPS and ICLR 2024 papers with public repositories using citation and repository activity signals (stars/forks). This ensures tasks are grounded in validated research with accessible implementations.

Stage 2: Experiment Procedure Extraction. Our pipeline employs two parallel extraction processes:

1. *Task Extraction*: We parse PDFs using OCR and multimodal processing to extract figures and tables. Retrieval-augmented, multi-pass prompts assemble research questions, methodologies, datasets, metrics, and constraints. Cross-document re-querying recovers missing details.

2. *Implementation Extraction*: A tool-augmented agent with access to PDFs, shell commands, and web search locates execution scripts, dependencies, and checkpoints. AST tracing converts scripts into step-by-step implementation requirements.

Stage 3: Verification & Refinement. We execute candidate scripts in clean Docker containers and compare outputs to paper expectations, iterating on failures. Human reviewers cross-check structured task content, and we mask answer-revealing files (README, scripts) via automated git operations to prevent shortcutting.

3.3 Dataset Statistics

EXP-Bench comprises 461 tasks from 51 NeurIPS/ICLR 2024 papers, totaling 12,737 fine-grained subtasks. Figure 3 shows the distribution across research areas: computer vision (CV), natural language processing (NLP), reinforcement learning (RL), and applications. Tasks span diverse methodologies including novel architectures, training procedures, evaluation protocols, and algorithmic innovations.

Each task provides a realistic research scenario with computational constraints matching typical academic settings: Ubuntu 24.04 environments with 4×A40 GPUs and 40-minute execution timeouts. This ensures tasks are both challenging and practically feasible.

3.4 Task Example

Figure 4 illustrates a representative EXP-Bench task from MogaNet research. The agent receives a research question about multi-order gated aggregation, a high-level methodology description, and a masked repository. Ground truth includes the complete experimental design, implementation diff showing the precise algorithm changes, and validated conclusions from executing the experiments.

Table 1: Average benchmark scores for various models when tested against various evaluation metrics. Popular Agents and LLMs perform poorly on EXP-Bench, showcasing its difficulty.

Agent	Model	D	I	E	I·E	C	All✓	All·E✓	#E
OpenHands	o3-mini	18.4	20.3	15.0	2.9	21.0	1.4	0.5	420
OpenHands	Claude-3.7 Sonnet	16.0	35.0	33.2	14.9	13.4	0.7	0.4	235
OpenHands	Amazon Nova Pro	18.2	19.5	26.8	0.0	15.7	0.0	0.0	56
OpenHands	Claude-3.5 Haiku	20.6	26.2	9.3	1.3	13.8	0.0	0.0	237
OpenHands	DeepSeek R1	6.8	10.0	0.7	0.0	2.4	0.0	0.0	140
IterativeAgent	Claude-3.5 Haiku	6.4	20.6	25.2	5.4	2.2	0.0	0.0	111
IterativeAgent	Amazon Nova Pro	0.1	10.0	18.1	0.0	0.3	0.0	0.0	215

4 Evaluation

4.1 Evaluation Protocol

Execution Environment. All experiments run in isolated Docker containers (Ubuntu 24.04, 4×A40 GPUs, 40-minute timeout) to ensure reproducible evaluation and prevent information leakage. A monitor detects disallowed behaviors including accessing original papers, using git checkout to reveal masked files, or generating fake data.

Multi-Metric Scoring. We employ a comprehensive evaluation protocol combining automated execution validation with LLM-based assessment:

- **Design (D) & Conclusion (C):** Evaluated by an o3-mini LLM judge comparing agent outputs to ground truth specifications and validated conclusions
- **Implementation (I):** Assessed by comparing agent-generated code to ground truth implementations, validated through execution testing
- **Execution (E):** Binary validation based on successful code execution producing expected outputs

Conjunctive Metrics. Beyond individual phase accuracy, we report conjunctive metrics that require simultaneous success: I·E (implementation *and* execution), All✓(design *and* implementation *and* conclusion), and All·E✓(complete success including execution).

4.2 Agent and Model Configurations

We evaluate seven agent-model combinations representing current state-of-the-art capabilities:

- **OpenHands [6]:** An open platform for AI software developers, tested with o3-mini, Claude-3.7 Sonnet, Amazon Nova Pro, Claude-3.5 Haiku, and DeepSeek R1
- **IterativeAgent:** A research-oriented agent with iterative refinement capabilities, tested with Claude-3.5 Haiku and Amazon Nova Pro

4.3 Main Results

Table 1 presents our primary findings across all agent-model combinations. The results reveal significant challenges in end-to-end research automation.

Individual Phase Performance. Agents achieve moderate success on isolated components, with the best individual performances being Implementation (35.0% for OpenHands + Claude-3.7 Sonnet) and Execution (33.2% for the same configuration). Design accuracy reaches 20.6% for OpenHands + Claude-3.5 Haiku, while Conclusion accuracy peaks at 27.5% for OpenHands + o3-mini in RL tasks.

Conjunctive Performance Collapse. The transition from individual to conjunctive metrics reveals a dramatic performance degradation. The best I·E (implementation *and* execution) performance is only 14.9%, achieved by OpenHands + Claude-3.7 Sonnet. Complete success rates (All·E✓) remain below 1% across all configurations, with the best performance being 0.5% for OpenHands + o3-mini.

Table 2: Average benchmark scores of various models and agents across select task categories; see supplement for complete list. Evaluation performed against EXP-Bench.

Category	Agent	Model	D	I	E	I·E	C	All✓	All✓·E
Applications	OH	Nova Pro	19.2	23.9	19.0	0.0	13.9	0.0	0.0
Applications	OH	o3-mini	9.0	8.0	0.0	0.0	8.3	0.0	0.0
Applications	OH	3.5 Haiku	19.2	24.5	8.3	5.6	8.3	0.0	0.0
Applications	OH	3.7 Sonnet	9.0	26.8	30.8	7.7	8.3	2.8	0.0
Applications	IA	Nova Pro	0.0	9.8	5.0	0.0	0.0	0.0	0.0
Applications	IA	3.5 Haiku	0.0	18.0	0.0	0.0	0.0	0.0	0.0
Applications	OH	DeepSeek R1	3.1	4.3	0.0	0.0	0.0	0.0	0.0
RL	OH	3.7 Sonnet	18.3	48.2	27.3	21.2	17.6	2.0	3.0
RL	OH	o3-mini	23.5	34.8	15.7	2.0	27.5	3.9	0.0
RL	OH	3.5 Haiku	27.7	41.4	11.5	0.0	17.6	0.0	0.0
RL	IA	3.5 Haiku	3.3	27.5	17.4	0.0	2.4	0.0	0.0
RL	OH	DeepSeek R1	5.0	10.3	0.0	0.0	2.0	0.0	0.0
RL	IA	Nova Pro	0.0	8.6	9.7	0.0	0.0	0.0	0.0
RL	OH	Nova Pro	17.9	28.9	0.0	0.0	13.7	0.0	0.0

Model and Agent Variations. Among the tested models, Claude-3.7 Sonnet demonstrates superior implementation and execution capabilities, while o3-mini shows relatively balanced performance across phases. DeepSeek R1 consistently underperforms across all metrics. OpenHands generally outperforms IterativeAgent, suggesting that the specialized development environment provides advantages for research tasks.

4.4 Performance by Research Category

Table 2 shows performance variation across research domains.

Reinforcement Learning Advantage. RL tasks demonstrate notably better performance, with OpenHands + Claude-3.7 Sonnet achieving 48.2% implementation accuracy and 21.2% I·E performance. This suggests that RL experiments may have more standardized implementation patterns or clearer evaluation protocols.

Applications Challenges. Tasks in the Applications category show consistently lower performance across all metrics, with most configurations achieving 0% complete success. This may reflect the diverse and domain-specific nature of application-oriented research.

4.5 Ablation Studies

Figure 5 presents ablation studies examining the relationship between evaluation strictness, cost, and performance.

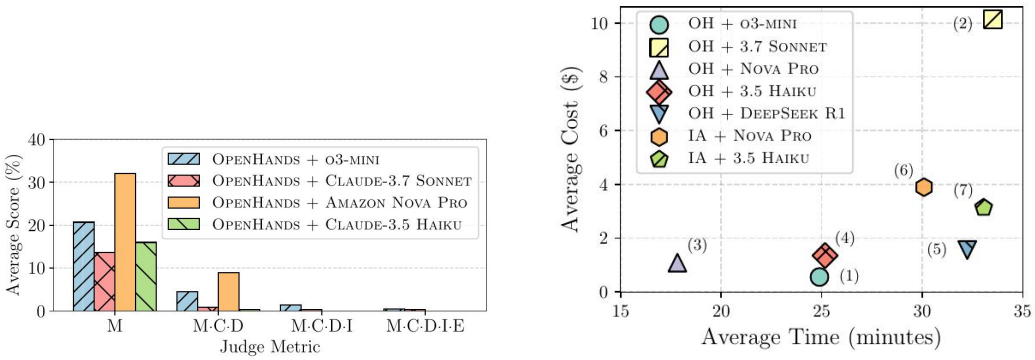
Evaluation Strictness. As evaluation becomes more stringent (from monitor-pass to conjunctive metrics), apparent performance drops sharply from an average of 20.6% to 0.2%. This highlights the importance of rigorous, multi-faceted evaluation for research automation tasks.

Cost-Time Trade-offs. The cost-time analysis reveals significant variation in computational efficiency across agent-model combinations. While higher-capability models often achieve better performance, the improvement may not justify the increased computational cost for many research scenarios.

4.6 Failure Analysis

Table 3 provides a detailed taxonomy of failure modes observed across all evaluations.

Implementation Failures. The most common failure mode is missing essential implementation components (39.7%), indicating that agents struggle to identify and implement all critical algorithmic elements. Environment and dependency configuration errors (29.4%) represent another major bottleneck, suggesting difficulties in managing complex computational environments.



(a) Stricter metrics reveal lower true correctness

(b) Cost-time trade-offs across agents

Figure 5: Ablation studies showing the impact of evaluation strictness and resource constraints on agent performance.

Table 3: Agents fail in diverse ways across different phases of experimentation; this table presents a simplified subset of common examples, measured across all agent and model evaluations.

Phase	Failure Type	Prevalence (%)
Design	Incomplete or Misclassified Design Variables	16.05
Design	Irrelevant Procedural Additions in Design	7.62
Implementation	Missing Essential implementation Components	39.71
Implementation	Incomplete Evaluation Metric Implementation	2.15
Implementation	Incomplete Data and Preprocessing Setup	1.83
Execution	Environment/Dependency Configuration Errors	29.38
Execution	Execution Script and File Errors	23.84
Execution	Missing Setup Script File	6.95
Execution	Tensor Operation Execution Error	3.22
Conclusion	Missing Conclusion Content	26.18
Conclusion	Incorrect Conclusion Interpretation	19.66
Conclusion	Extraneous Details in Conclusion	7.77
Conclusion	Incorrect Numeric Conclusion	3.21

Design and Conclusion Issues. Design phase failures include misclassified variables (16.0%) and irrelevant procedural additions (7.6%). Conclusion phase problems encompass missing content (26.2%) and incorrect interpretations (19.7%), highlighting challenges in scientific reasoning and result interpretation.

Execution Bottlenecks. Script and file errors (23.8%) along with missing setup files (7.0%) indicate that agents struggle with the practical aspects of experiment execution, even when the algorithmic implementation is correct.

5 Discussion

5.1 Implications for AI Research Automation

Our results reveal a fundamental gap between current AI capabilities and the requirements for autonomous research. While agents demonstrate competency in isolated tasks—achieving up to 35% implementation accuracy—the conjunctive nature of research demands simultaneous success across multiple complex phases. The dramatic performance collapse from individual metrics ($\sim$20%) to complete success ($\sim$0.5%) suggests that research automation requires more than incremental improvements in existing capabilities.

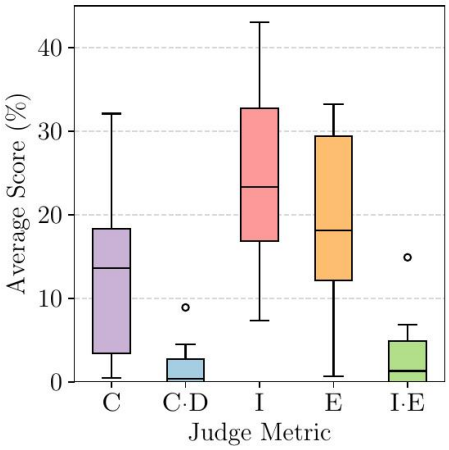


Figure 6: Stability analysis showing consistency of agent performance across multiple evaluation runs, demonstrating the reliability of our evaluation protocol.

The Conjunctive Challenge. Real research success requires the conjunction of multiple capabilities: sound experimental design *and* correct implementation *and* successful execution *and* valid conclusions. Our results demonstrate that current agents struggle with this conjunctive requirement, even when individual components show moderate success. This suggests that research automation may require fundamentally different approaches than those optimized for isolated tasks.

Systematic vs. Random Failures. The failure taxonomy reveals systematic rather than random limitations. Missing implementation components (39.7%) and environment configuration errors (29.4%) suggest specific areas where current agents lack necessary capabilities or knowledge. This provides clear directions for future development.

5.2 Methodological Insights

Benchmark Design Lessons. EXP-Bench demonstrates the importance of execution validation in AI evaluation. Traditional benchmarks that rely solely on static code analysis or LLM judgment miss critical failures that only emerge during execution. Our Docker-based validation reveals that many implementations that appear correct fail when executed in realistic environments.

Grounding in Real Research. By anchoring tasks in peer-reviewed publications and their implementations, EXP-Bench provides higher fidelity evaluation than synthetic benchmarks. This grounding ensures that tasks reflect real research complexity while providing validated ground truth for assessment.

Evaluation Reliability. Figure 6 shows that our evaluation protocol produces consistent results across multiple runs, validating the reliability of our assessment methodology. This stability is crucial for meaningful comparison of agent capabilities.

5.3 Future Directions

Targeted Capability Development. Our failure analysis suggests specific areas for improvement: better environment management tools, more sophisticated implementation planning capabilities, and enhanced scientific reasoning for conclusion generation. Rather than general capability improvements, research automation may benefit from targeted development in these areas.

Hybrid Human-AI Workflows. Given the conjunctive challenges revealed by EXP-Bench, practical research automation may require hybrid approaches where AI handles specific components while humans provide oversight and integration. Understanding the optimal division of labor represents an important research direction.

Benchmark Evolution. EXP-Bench provides a foundation for evaluating research automation, but the benchmark should evolve with advancing capabilities. Future versions could incorporate more recent research areas, longer experimental workflows, or multi-paper synthesis tasks.

5.4 Limitations

Our evaluation focuses on implementation-heavy AI research tasks and may not generalize to all research domains. The 40-minute execution timeout, while realistic, may exclude longer experiments. Additionally, our LLM judge evaluation, while validated, could introduce biases in design and conclusion assessment. Future work should explore alternative evaluation methodologies and expand domain coverage.

6 Conclusion

We introduce EXP-Bench, the first benchmark to evaluate AI agents on complete, executable AI research experiments. Through comprehensive evaluation of seven agent-model combinations on 461 research tasks extracted from peer-reviewed publications, we reveal significant limitations in current research automation capabilities.

Our key findings demonstrate that while AI agents achieve moderate success on individual research components—with best performances of 35.0% implementation accuracy and 33.2% execution success—they struggle dramatically with end-to-end experimental workflows. Complete success rates remain below 1% across all tested configurations, highlighting the conjunctive challenge inherent in research automation.

The systematic failure analysis reveals specific bottlenecks: missing essential implementation components (39.7%), environment configuration errors (29.4%), and incorrect scientific conclusions (19.7%). These patterns suggest that research automation requires targeted capability development rather than general improvements in existing skills.

EXP-Bench provides a rigorous, execution-validated testbed for advancing AI research automation. Our benchmark contributes: (1) the first evaluation framework for complete research workflows, (2) a scalable pipeline for extracting executable tasks from scientific literature, (3) comprehensive assessment revealing current limitations, and (4) a detailed roadmap for future development through systematic failure analysis.

The transition from component-wise AI capabilities to integrated research systems represents a critical challenge for the field. EXP-Bench offers both a measurement framework for progress and insights into the fundamental requirements for autonomous scientific experimentation. As AI capabilities continue advancing, EXP-Bench provides a valuable benchmark for tracking progress toward genuine research automation.

Future work should address the specific failure modes we identify, explore hybrid human-AI research workflows, and extend evaluation to broader scientific domains. The ultimate goal—AI systems capable of conducting rigorous, reproducible research—remains challenging but increasingly important as scientific complexity grows and research demands scale.

References

[1] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang. Researchagent: Iterative research idea generation over scientific literature with large language models. *arXiv preprint arXiv:2404.07738*, 2024.

[2] A. Ghafarollahi and M. J. Buehler. Sciagents: Automating scientific discovery through multi-agent intelligent graph reasoning, 2024.

[3] T. Liu, C. Xu, and J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems, 2023.

[4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023.

[5] Meta. Code Llama: Open Foundation Models for Code. 2023. https://ai.meta.com/research/publications/code-llama-open-foundation-models-for-code/.

[6] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.